PERFORMANCE STUDY

성능 테스트
Performance Testing.

성능 테스트 스터디에서,
오늘은 부하 테스트를 중심으로 본다.

SECTION 01

부하 테스트란?

> 시스템이 **예상되는 트래픽을 안정적으로 처리**할 수 있는지
> 검증하는 테스트

→ 성능 테스트의 여러 영역 중, **예상 트래픽을 안정적으로 처리하는 범위를 확인하는 과정.**

SECTION 02 · 왜 필요한가

트래픽 변동은 연쇄적으로 이어진다.

- 이벤트 오픈
- 특정 시간대 트래픽 집중
- 마케팅 캠페인
- 티켓팅 오픈
- 대학 수강신청

트래픽 증가 → 처리 지연 → 서비스 중단 → 사용자 이탈 → **신뢰도 하락**

02
우리 주변의 실제 사례

사전에 측정하지 않으면 실제 운영에서야 한계가 보인다.

01

대학 수강신청

명지대는 L4 스위치 과부하로 일정이 연기됐고, 이화여대는 자정까지 변경 기간을 연장했지만 재차 중단. 건국대는 한 학기 4번 중 정상 진행이 1번에 그쳤다.

CAUSE 동시 접속 집중

IMPACT 신청 지연·일정 변경

매 학기 반복

02

콘서트·뮤지컬 티켓팅

인터파크는 대기열로 트래픽을 분산 처리. 예스24는 대기열 없이 일괄 입장하는 구조로 로그인 해제, 예매 버튼 누락 등 정합성 이슈가 발생했다.

CAUSE 짧은 시간 피크

IMPACT 대기열·예매 정합성

동시 접속 집중

03

예스24 서비스 중단

2025년 6월, 랜섬웨어로 도서·티켓·eBook 등 주요 온라인 서비스가 중단됐고 일부 서비스는 며칠 뒤 재개됐다. 부하 사례는 아니지만, 시스템 정지가 만드는 연쇄 영향을 보여준다.

CAUSE 랜섬웨어 장애

IMPACT 도서·티켓 서비스 중단

2025.06 · 운영 리스크

02

왜 그냥 큰 서버를 쓰지 않을까?

비용과 안정성의 최적 균형점.

✕

무조건 큰 서버

인프라 비용은 트래픽 대비 계속 증가하고, 피크 시간을 기준 잡으면 대부분 시간이 낭비된다. 오토스케일링 기준점도 결국 측정값이 있어야 정할 수 있다.

OVER-PROVISIONING

비용	높음
활용률	낮음

✓

최소 비용으로 안정 확보

부하 테스트는 **"안정성을 확보하는 최소 비용 구조"**를 찾기 위한 과정이다. 한계를 알아야 비용을 줄이면서도 안정적으로 운영할 수 있다.

MEASURED SCALING

측정값	임계점	스케일 기준

목적에 따라 4가지로 나뉜다.

종류	목적	활용 시점
Load Test	예상 트래픽에서 안정적으로 동작하는지 확인	이번 스터디의 주제
Stress Test	한계치 초과 시 시스템이 어떻게 무너지는지	내구 한계 파악
Spike Test	갑작스러운 트래픽 급증에도 안정적으로 응답하는지	순간 급증 대응 확인
Soak Test	장시간 운영 시 메모리 누수·성능 저하 발견	장기 안정성 검증

LOAD

STRESS

SPIKE

SOAK

k6는 이 모든 유형을 `options` 시나리오 코드로 표현할 수 있다.

04
SECTION 04

VU. 가상 사용자.

VU = 사용자 행동을 반복 실행하는 시뮬레이션된 사용자 1명.

같은 가상 사용자라도 도구마다 실행 모델이 다르다.

k6.options.js

```
export const options = { vus: 50, // 가상 사용자 50명 duration: '30s' // 30초 동안 유지 };
```

k6

Scenario 로그인 · 조회 · 클릭 → **VU** 사용자 역할 → **k6 Runtime** 실행 엔진 → **Target** 요청 처리

핵심: VU와 OS Thread가 1:1로 매핑되지 않는다.

JMeter

Scenario 테스트 플랜 → **Thread Group** 사용자 그룹 → **Thread** 작업자 단위 → **Target** 요청 처리

핵심: Thread 기반으로 동시 사용자를 표현한다.

nGrinder

Scenario Groovy 스크립트 → **Controller** 테스트 지시 → **Agent** 부하 생성 → **Thread** 작업자 단위 → **Target** 요청 처리

핵심: Controller가 지시하고 Agent가 분산 부하를 만든다.

05

SECTION 05 · 핵심 지표

반드시 봐야 할 3가지 지표.

처리량

TPS

01

초당 처리 가능한 요청 수. 시스템의 최대 처리 용량을 가늠하는 1차 지표.

신호 증가 후 정체

의미 Capacity 도달

지연 시간

Latency

02

요청 → 응답까지 걸리는 시간. 평균이 아닌 **P95·P99**로 봐야 한다.

신호 P99 급상승

의미 큐 대기 증가

실패율

Error Rate

03

실패 요청 비율. 에러가 발생하기 시작하는 지점이 시스템 한계의 신호다.

신호 5xx·Timeout

의미 한계 초과

05·1

처리량 · TRANSACTIONS PER SECOND

① TPS.

```
TPS ≈ VU 수 ÷ (응답시간 + 대기시간) 예) VU=100, 응답시간=200ms, 대기시간
=800ms → TPS ≈ 100 ÷ 1.0 = 100 TPS
```

실제 TPS는 VU 수, 응답시간, 대기시간, 시나리오 구조에 함께 영향을 받는다.

- VU가 증가하면 TPS도 **증가**한다.
- 그러나 어느 순간 증가가 **멈춘다.**
- 이 지점이 → **시스템의 최대 처리 용량 (Capacity).**

05·2

지연 시간 · 평균의 함정

② Latency.

평균만 보면 왜곡된다.

요청 99개 → 100ms

요청 1개 → 3,000ms

평균 = 약 130ms

평균 연봉 예시

직원 9명 × 3천만 원

CEO 1명 × 30억 원

평균 연봉 ≈ 3억 원

대다수 직원의 상황은 가려진다.

겉보기엔 빠른 서비스이지만,

일부 사용자는 느린 응답을 경험한다.

- **P95** — 100개 중 95개는 이 시간 이하. "대부분의 사용자 경험"
- **P99** — 100개 중 99개는 이 시간 이하. "최악에 가까운 경험" · 병목 감지
- 실제 사용자 경험은 **느린 요청이 좌우**한다.
- 평균은 "전체 속도", P95·P99는 **"실제로 겪는 느림."**

P99는 평균보다 먼저 흔들린다.

구간	평균	P95	P99
① 초기	안정	안정	안정
② 임계점 도달	거의 유지	서서히 ↑	급격히 ↑
③ 한계 초과	약간 ↑	크게 ↑	급격히 ↑↑

대부분의 서버는 내부적으로 **큐(Queue) 기반**으로 요청을 처리한다. 처리 한계를 넘으면 요청이 쌓이면서 대기 시간이 빠르게 증가한다.

05·3

실패율 · THE LIMIT SIGNAL

③ Error Rate.

핵심 의미

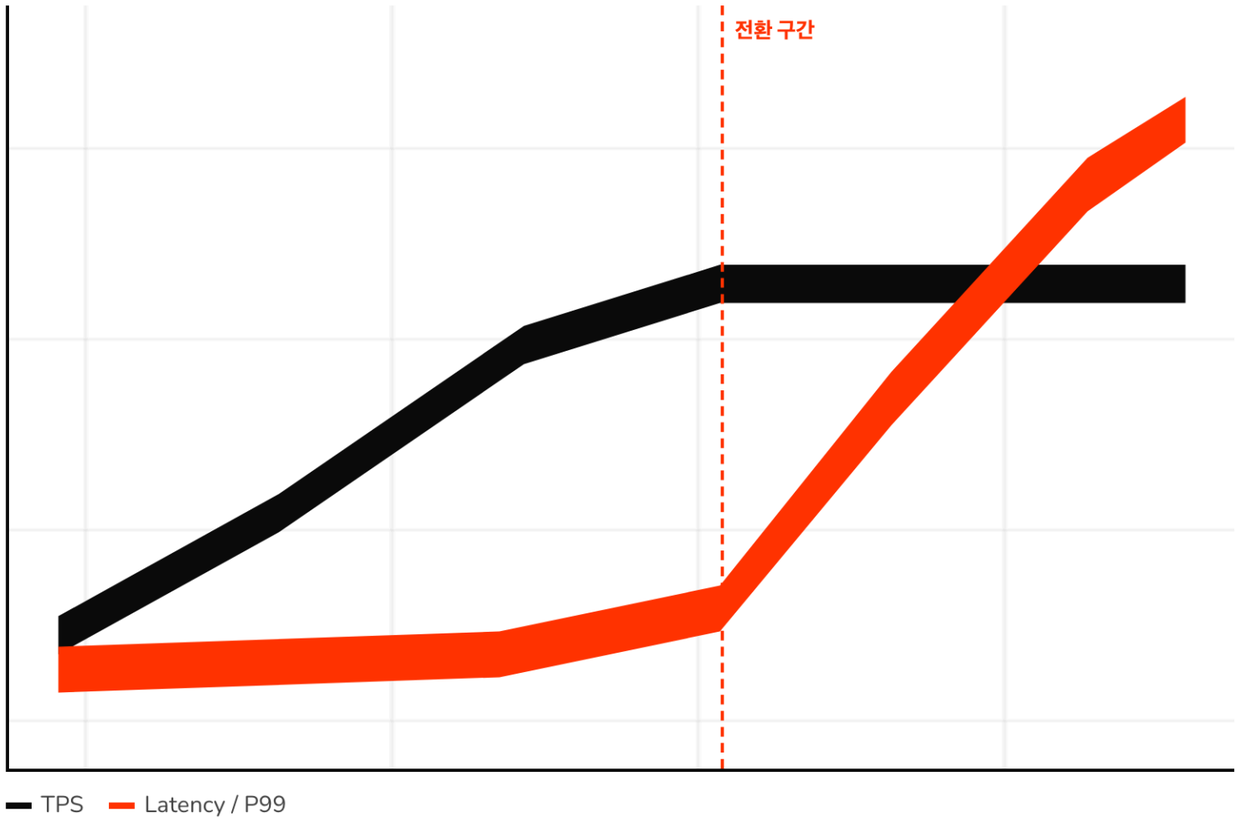
에러가 발생하는 시점은
시스템 한계의 신호.

반드시 체크할 항목.

- **HTTP 5xx** — 서버 에러 응답
- **Timeout** — 응답이 시간 내 안 옴
- **Connection 실패** — 연결 자체가 끊어짐

시스템이 안정적으로 유지되는 전환 구간을 찾는 것.

CAPACITY TRANSITION



01

VU↑→TPS↑

사용자가 늘어나면 처리량도 비례해서 증가하는 안정 구간.

02

TPS 정체

어느 순간 처리량 증가가 멈춘다 — Capacity 도달.

03

Latency↑(P99)

큐가 쌓이면서 지연 시간이 증가하고, P99가 평균보다 먼저 흔들린다.

04

Error 발생

한계 초과. 에러 발생은 시스템이 더 이상 못 버틴다는 신호.

부하 생성 서버는 분리해서 측정한다.

LOAD GENERATOR

k6

부하 생성 전용 서버. 트래픽을 만든다.

→ → →

TARGET SERVER

Spring Boot

측정 대상. 순수한 성능만 평가.

같은 서버에서 k6와 대상 앱을 동시에 실행하면, 측정된 Latency에 **실제 처리 지연과 부하 생성기의 자원 경합**이 함께 섞인다. 서버 성능과 도구 영향을 구분하기 어려워진다.

k6 · JMeter · nGrinder.

항목	K6	JMETER	NGRINDER
작성 방식	JavaScript 코드	GUI (XML 저장)	웹 UI + Groovy
기반 언어	Go 런타임	Java / JVM	Java + Groovy
실행 방식	CLI	GUI / CLI	웹 UI (Controller-Agent)
메모리 / VU	VU당 ~1-5 MB	상대적으로 높음	JVM 기반 (Agent 분산)
동시 사용자	대규모 VU 스크립트·장비 영향	스레드 기반 제약	대규모 VU Agent 분산 확장
동시성 모델	Goroutine + Event Loop	Thread per VU	Thread per VU Agent별 분산
분산 부하	k6 Operator (k8s)	JMeter 서버 분산	Controller-Agent 네이티브
프로토콜	HTTP, WebSocket, gRPC 중심	DB·JMS·SMTP 등 광범위	HTTP 중심
CI/CD	CI/CD 친화적	별도 설정 필요	가능하지만 번거로움
Git 협업	코드 기반	XML 추적 어려움	코드 + 웹 UI
시각화	Grafana + Prometheus	HTML 리포트 + 플러그인	웹 UI 내장 + CSV/tgz
학습 곡선	JS 알면 빠름	GUI는 쉬움 고급은 플러그인 의존	설정 복잡 진입장벽 있음
첫 출시	2017년	1998년	2011년 네이버 사내 도구

options 같은 점선 밑줄 키워드는 hover 또는 클릭으로 설명을 볼 수 있다.

08

어느 도구를 언제 쓰나?

상황에 맞는 도구 선택.

PERFORMANCE AS CODE

k6

JS 기반 코드. Go 런타임 기반으로 대규모 테스트에 효율적. Grafana + Prometheus 연동이 강력. 단점은 GUI 없음, JDBC/SMTP 약함.

BEST	FLOW	TEAM
Web API	CI/CD	개발팀

→ 일반적인 웹 API + CI/CD 자동화

LEGACY STANDARD

JMeter

1998년부터 GUI로 누구나. **JDBC·FTP·JMS·SMTP·LDAP** 등 광범위한 프로토콜 지원. 단점은 무거움, XML로 Git 추적 곤란, 코드 리뷰 사실상 불가.

BEST	FLOW	TEAM
다중 프로토콜	GUI 작성	QA 협업

→ DB·메일·메시지큐 + GUI 작업

DISTRIBUTED NATIVE

nGrinder

네이버 사내 도구 → 2011 오픈소스. **Controller + Agent** 네이티브 분산 구조. 웹 UI에서 모니터링까지 한 곳. 단점은 진입 장벽 높음.

BEST	FLOW	TEAM
분산 부하	웹 콘솔	사내 공용

→ 수십만 VU 대규모 분산 + 사내 공용

오늘 다룬 부하 테스트는 ~~서버가 언제 한계에 부딪히는지 보는 테스트~~가 아니라, **어디까지 안정적으로 처리할 수 있는지 찾아가는 과정이다.**

TPS

처리량

최대 처리 용량 파악

P99

지연 시간

병목 구간 조기 감지

Error Rate

실패율

한계 초과 시점 감지

지표를 읽는 눈 + 상황에 맞는 도구 선택 = 진짜 실력